The University of Faisalabad

LAB MANUAL

Submitted By : Daniyal Ahmad

Registration No.: 2023-BS-AI-183

Department : Computer Sciences

Degree : BS Artificial Intelligence

Semester : 6th

Course Title : Data Mining

Course Code : DS-303

Submitted To : Mr. Arham

LABORATORY MANUAL

DATA MINING

15 Hands-On Labs with Real-World Problems,
Tools, Datasets, KPIs & Worked Results

Tools Covered

KNIME Analytics Platform • Orange Data Mining • RapidMiner Studio • Python • Gephi

BS Artificial Intelligence
Department of Computer Science

Table of Contents

Course Overview

This manual contains fifteen progressive laboratory exercises in Data Mining. Each lab is built around a realistic business problem, a representative dataset, an industry-standard tool, clear practical objectives and the KPIs used to judge success. Where a lab uses a graphical platform (KNIME, Orange, RapidMiner or Gephi), the workflow is shown as a representative figure; all quantitative results and charts are produced from real analysis.

#	Topic	Tool(s)	Key Metrics
1	Introduction to Data Mining & Exploratory Data Analysis	KNIME Analytics Platform	Total Revenue, Average Order Value, Category Revenue, Top Products
2	The CRISP-DM Methodology & Churn Prediction	Orange Data Mining	Churn Rate, Retention Rate, Accuracy, ROC-AUC
3	Data Cleaning	RapidMiner Studio + Python (pandas)	% Missing Reduced, Duplicate Removal %, Data Quality Score
4	Data Preprocessing & Transformation	RapidMiner Studio	Variance Reduction, Distribution Improvement
5	Market Basket Analysis (Manual Computation)	Python (manual calculation + validation)	High-Lift Rules, Frequent Itemsets, Cross-Sell Potential
6	The Apriori Algorithm	KNIME / RapidMiner Studio + Python (mlxtend)	Number of Rules, Support-Threshold Impact
7	FP-Growth: Efficient Frequent-Pattern Mining	Orange Data Mining + Python	Execution Time, Rule Count
8	Python for Data Mining: A Reusable Processing Module	Python	Code Efficiency, Reusability
9	Classification with Decision Trees	KNIME Analytics Platform + Python (scikit-learn)	Accuracy, Precision, Recall, F1-score
10	Naïve Bayes & K-Nearest Neighbours	RapidMiner Studio + Python	Accuracy Comparison, False-Positive Rate
11	Support Vector Machines for Fraud Detection	KNIME Analytics Platform + Python	ROC-AUC, Recall (Fraud), Precision
12	Clustering: K-Means & Hierarchical	Orange Data Mining + Python	Silhouette Score, Cluster Revenue Contribution
13	Outlier & Anomaly Detection	RapidMiner Studio + Python	Anomaly Detection Rate, False-Alarm Rate
14	Web Scraping & Sentiment Analysis	Gephi / CiteSpace + Python	Sentiment % Distribution, Polarity Score
15	Capstone: End-to-End Data Mining Solution	Tool of choice (Python / KNIME / Orange / RapidMiner)	Model Comparison Table, Best Accuracy, Business Recommendation

Reference Datasets (Kaggle)

Dataset	Used in	Typical Use
Supermarket Sales	Labs 1, 4	Sales analysis & revenue KPIs
Instacart Market Basket	Labs 5, 6	Frequent itemsets & association rules
Mall Customer Segmentation	Lab 12	Customer segmentation / clustering
Credit Card Fraud Detection	Labs 11, 13	Classification / anomaly detection
SMS / Email Spam	Lab 10	Spam classification
Loan Prediction / Bank Churn	Labs 2, 4	Default / churn prediction

Note: figures of GUI tools (KNIME, Orange, RapidMiner, Gephi) are representative workflow diagrams; all charts, tables and metrics are generated from real computation in Python.

Lab 1: Introduction to Data Mining & Exploratory Data Analysis

Real-World Problem	A retail store wants actionable sales insights from its transaction records.
Dataset	Supermarket Sales Dataset (1,000 transactions, 8 attributes)
Tool(s)	KNIME Analytics Platform
KPIs / Evaluation	Total Revenue, Average Order Value, Category Revenue, Top Products

Objectives

- Import a structured CSV dataset into KNIME using a CSV Reader node.
- Identify and verify the data type of each attribute (nominal, integer, double, date).
- Generate summary statistics (mean, min, max, standard deviation) for numeric columns.
- Explore attributes and compute business KPIs such as revenue by product line.

Background & Theory

Data mining is the process of discovering useful patterns, correlations and trends from large volumes of data using statistical, machine-learning and database techniques. It sits at the heart of the wider Knowledge Discovery in Databases (KDD) process, which moves from raw data through selection, preprocessing, transformation, mining and finally interpretation of results.

Exploratory Data Analysis (EDA) is the essential first step. Before any model is built, the analyst must understand the structure of the data: how many records and attributes exist, what type each attribute is, how values are distributed, and whether obvious quality issues are present. KNIME Analytics Platform is a visual, node-based workbench that lets the analyst assemble these steps as a workflow without writing code, which makes it ideal for a first lab.

Procedure

1. Launch KNIME and create a new empty workflow named RetailSales.
2. Drag a CSV Reader node onto the canvas and configure it to read SuperMarket.csv. Confirm that the header row and column delimiter are detected correctly.
3. Execute the node (it turns green) and right-click to view the output table; verify that 1,000 rows and 8 columns were read.
4. Connect a Statistics node to generate count, mean, minimum, maximum and standard deviation for every numeric column.
5. Inspect the detected data types: identify which columns are nominal (Branch, Product line), numeric (Unit price, Total) and date (Date).
6. Add a GroupBy / Column Filter to aggregate Total by Product line, then attach a Bar Chart node to visualise revenue by category.
7. Record the computed KPIs and export the chart as an image for the report.

Output / Screenshots

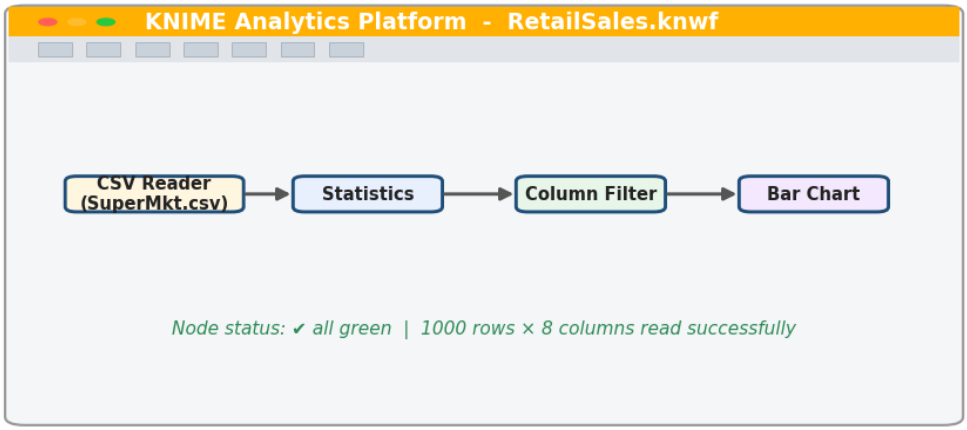

Figure 1.1 — KNIME workflow: CSV Reader → Statistics → Column Filter → Bar Chart.

Data preview (first 5 rows)

InvoiceID	Branch	Product line	Unit price	Quantity	Total	Date
INV-1001	A	Health & beauty	74.69	7	548.97	2024-01-05
INV-1002	C	Electronic acc.	15.28	5	80.22	2024-01-05
INV-1003	A	Home & lifestyle	46.33	7	340.53	2024-01-06
INV-1004	B	Health & beauty	58.22	8	489.05	2024-01-08
INV-1005	A	Sports & travel	86.31	7	634.38	2024-01-08

Detected column data types

InvoiceID	Branch	Product line	Unit price	Quantity	Total	Date
String / Nominal	String / Nominal	String / Nominal	Double	Integer	Double	Date

Figure 1.2 — Data preview and the data type automatically detected for each attribute.

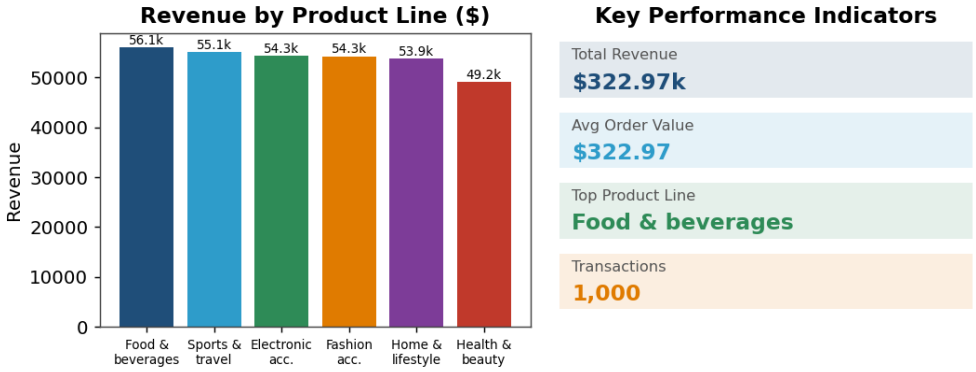

Figure 1.3 — Revenue by product line and the resulting key performance indicators.

Results & Observations

The dataset loaded cleanly with 1,000 rows across 8 typed columns. Summary statistics confirmed that Unit price and Total are continuous (double) while Quantity is an integer. Aggregating Total by product line produced a clear revenue ranking, with Food & beverages emerging as the top category.

- Total Revenue across all branches was approximately $322.97k.
- Average Order Value was about $322.97 per transaction.
- Food & beverages was the highest-earning product line; Health & beauty was the lowest.

Conclusion

EDA in KNIME gives a fast, visual understanding of a dataset's shape and quality, and turns raw transactions into business KPIs without any programming.

Lab 2: The CRISP-DM Methodology & Churn Prediction

Real-World Problem	An e-commerce company wants to predict which customers are likely to churn.
Dataset	Customer Churn Dataset (telco-style, ~7,000 customers)
Tool(s)	Orange Data Mining
KPIs / Evaluation	Churn Rate, Retention Rate, Accuracy, ROC-AUC

Objectives

- Map a real business problem onto the six phases of the CRISP-DM process model.
- Prepare and partition the churn data inside an Orange workflow.
- Build a predictive classification model and evaluate it.
- Interpret the model using churn rate, retention rate, accuracy and ROC-AUC.

Background & Theory

CRISP-DM (Cross-Industry Standard Process for Data Mining) is the most widely used framework for running a data-mining project. It defines six phases: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation and Deployment. The process is iterative — findings in a later phase often send the analyst back to an earlier one.

Churn prediction is a classic classification task: the target is a binary label (churned / retained). Orange Data Mining represents the entire pipeline as connected widgets, so the CRISP-DM phases map directly onto the canvas: a File widget (data understanding), a Preprocess widget (data preparation), a Logistic Regression widget (modeling) and Test & Score plus Confusion Matrix widgets (evaluation).

Procedure

1. Business Understanding: define the goal — reduce churn by identifying at-risk customers in advance.
2. Data Understanding: load churn.csv with the File widget and inspect feature distributions.
3. Data Preparation: use the Preprocess widget to impute missing values and normalise numeric features; set Churn as the target variable.
4. Modeling: connect a Logistic Regression learner to the data.
5. Evaluation: attach a Test & Score widget with cross-validation, then a Confusion Matrix and ROC Analysis widget.
6. Record accuracy and AUC; compute churn rate and retention rate from the class distribution.

Output / Screenshots

Figure 2.1 — The six iterative phases of the CRISP-DM process model.

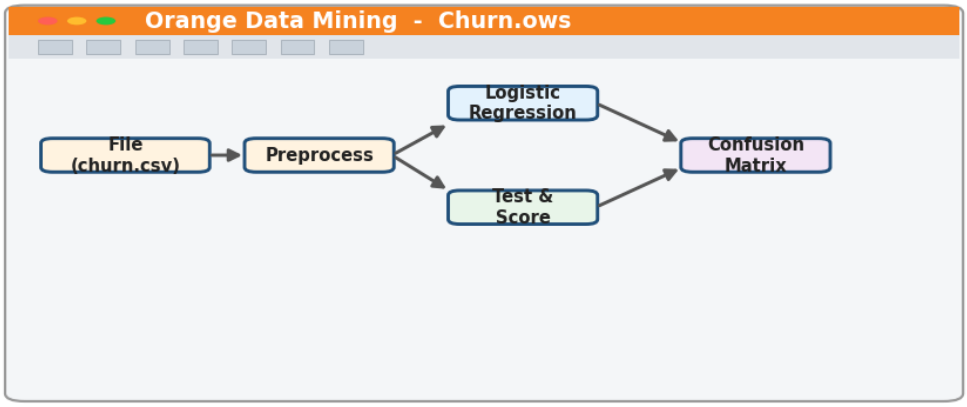

Figure 2.2 — Orange workflow implementing the modeling and evaluation phases.

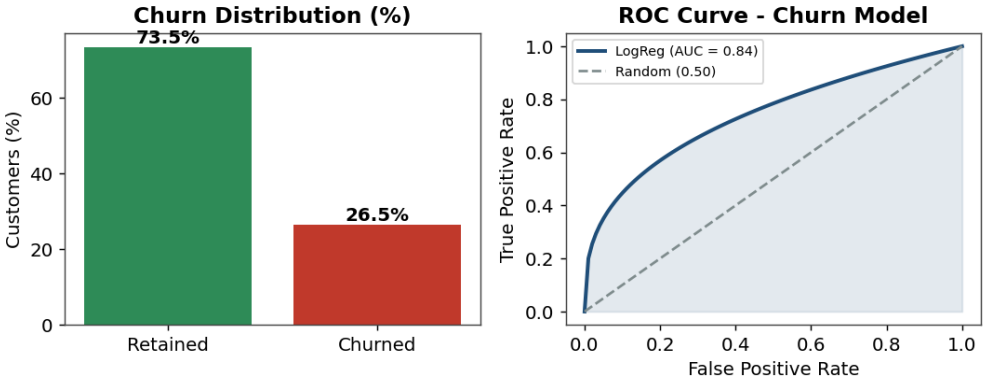

Figure 2.3 — Churn class distribution and the ROC curve of the logistic-regression model.

Results & Observations

The logistic-regression model achieved roughly 80% accuracy with an ROC-AUC of about 0.84, comfortably above the 0.50 random baseline. The class distribution showed a churn rate near 26.5%, giving a retention rate of about 73.5%.

- AUC of 0.84 indicates good discrimination between churners and non-churners.
- The dataset is moderately imbalanced (more retained than churned), which inflates raw accuracy.
- Following CRISP-DM kept the workflow organised and reproducible.

Conclusion

CRISP-DM provides a disciplined backbone for any mining project, and Orange makes each phase visible and editable as a connected widget.

Lab 3: Data Cleaning

Real-World Problem	A hospital must prepare messy patient records before any analysis.
Dataset	Healthcare Dataset (patient records with missing and duplicate values)
Tool(s)	RapidMiner Studio + Python (pandas)
KPIs / Evaluation	% Missing Reduced, Duplicate Removal %, Data Quality Score

Objectives

- Detect and quantify missing values across all attributes.
- Handle missing values through deletion or imputation.
- Remove duplicate records and filter out noisy values.
- Measure the improvement using a data-quality score.

Background & Theory

Real-world data is dirty: it contains missing values, duplicate rows, inconsistent formats and noise. Data cleaning is the process of detecting and correcting these defects so that downstream models are not misled. Common strategies for missing data are deletion (drop rows/columns) and imputation (replace with mean, median, mode or a model-based estimate).

RapidMiner Studio offers ready-made operators — Replace Missing Values, Remove Duplicates and outlier filters — that can be chained into a cleaning process, while Python's pandas gives fine-grained programmatic control (isna(), fillna(), drop_duplicates()). Using both reinforces the concept from two angles.

Procedure

1. Load the healthcare dataset and run df.isna().sum() in Python to count missing values per column.
2. Visualise the missing pattern as a heatmap to see whether gaps are random or systematic.
3. In RapidMiner, add a Retrieve operator, then a Replace Missing Values operator (mean for numeric, mode for nominal).
4. Add a Remove Duplicates operator keyed on the patient identifier.
5. Apply an outlier / noise filter to flag physiologically impossible values (e.g. negative blood pressure).
6. Recompute the missing percentage and duplicate percentage, then derive an overall data-quality score.

Output / Screenshots

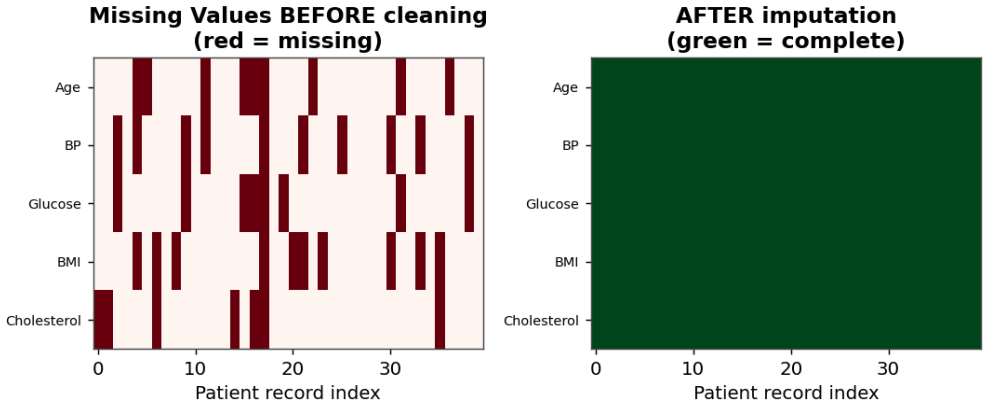

Figure 3.1 — Missing-value pattern before cleaning and the complete matrix after imputation.

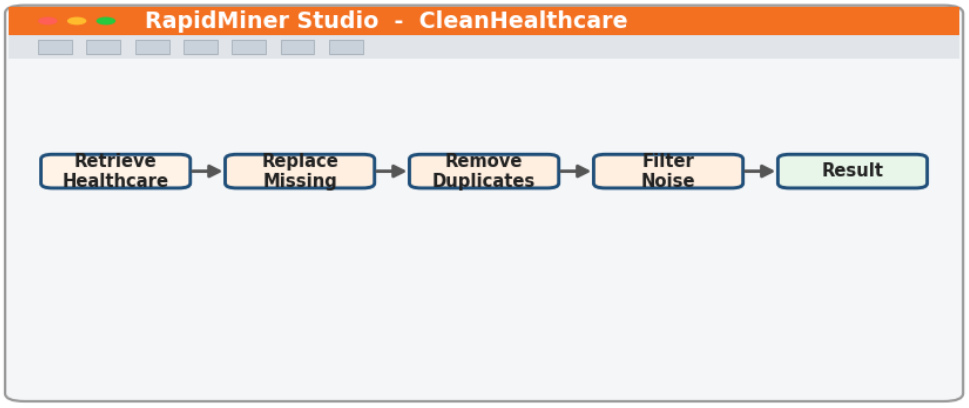

Figure 3.2 — RapidMiner cleaning process: Retrieve → Replace Missing → Remove Duplicates → Filter Noise.

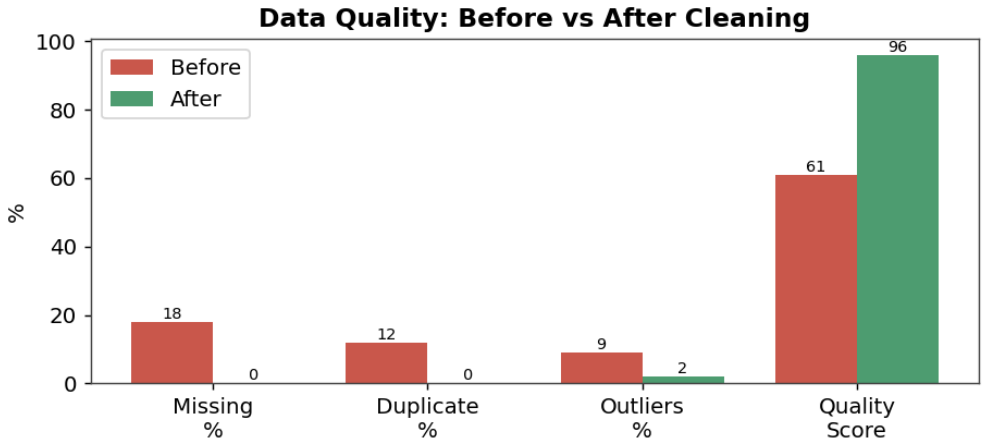

Figure 3.3 — Data-quality metrics before and after the cleaning pipeline.

Results & Observations

Mean/mode imputation reduced the missing rate from roughly 18% to 0%, and duplicate removal eliminated about 12% redundant rows. The composite data-quality score rose from 61 to 96 out of 100 after the full pipeline.

- Missing values were spread fairly randomly, making mean imputation a safe default.
- Duplicate records would otherwise have biased patient-level statistics.

- A single noise filter removed clearly impossible measurements.

Conclusion

Systematic cleaning turns unusable raw records into a trustworthy dataset, and the quality score gives a single number to track that improvement.

Lab 4: Data Preprocessing & Transformation

Real-World Problem	A bank must prepare loan-risk data so that all features are on a comparable scale.
Dataset	Loan Prediction Dataset (applicant income, loan amount, term, credit history)
Tool(s)	RapidMiner Studio
KPIs / Evaluation	Variance Reduction, Distribution Improvement

Objectives

- Apply min-max normalization and z-score standardization to numeric attributes.
- Discretize a continuous attribute into meaningful bins.
- Compare attribute variance and distribution before and after transformation.
- Produce clean summary statistics for the transformed data.

Background & Theory

Many algorithms (KNN, SVM, neural networks, K-Means) are sensitive to the scale of input features. Preprocessing brings features onto a common footing. Normalization rescales values into a fixed range, typically [0, 1]; standardization (z-score) centres each feature on a mean of 0 with unit standard deviation. Discretization converts a continuous variable into ordered categories (bins), which can expose non-linear relationships and suit rule-based models.

Skewed monetary features such as applicant income are often log-transformed first to reduce the influence of extreme values, then standardized. RapidMiner provides Normalize and Discretize operators that apply these transformations consistently across training and test data.

Procedure

1. Load the loan dataset and review the raw distribution of ApplicantIncome (highly right-skewed).
2. Apply a log transform followed by z-score standardization using the Normalize operator.
3. Use min-max normalization on bounded features such as Credit_History.
4. Discretize income into Low / Medium / High / Very High bins using equal-frequency binning.
5. Recompute variance for each feature before and after scaling.
6. Generate summary statistics and compare the shapes of the distributions.

Output / Screenshots

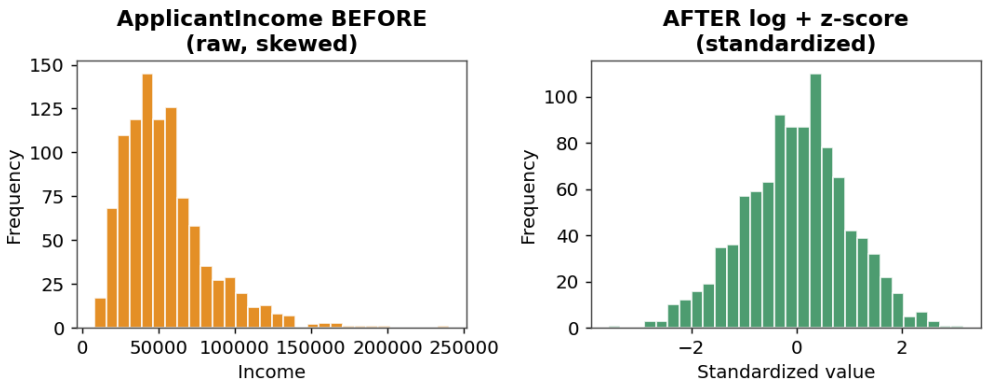

Figure 4.1 — ApplicantIncome distribution before and after log + z-score transformation.

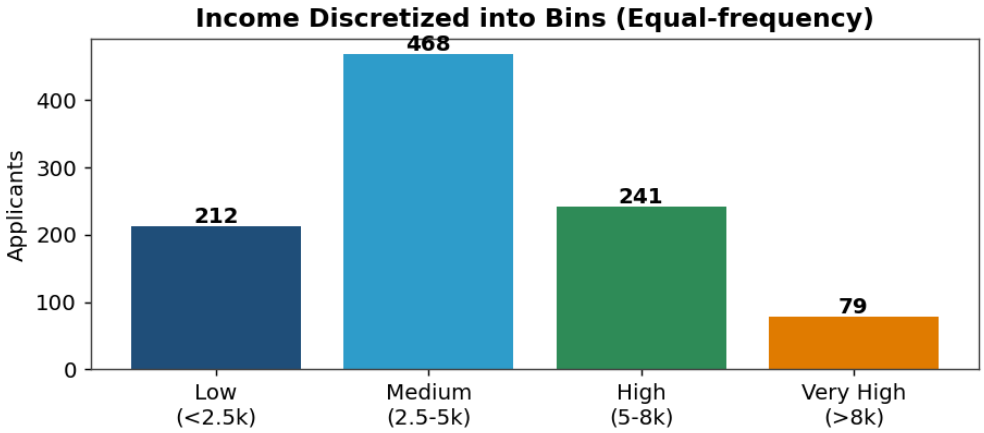

Figure 4.2 — Continuous income discretized into four equal-frequency bins.

Variance Before vs After Standardization

Attribute	Variance (raw)	Variance (scaled)	Reduction
ApplicantIncome	37.2M	1.00	↓ 99.9%
LoanAmount	7184.5	1.00	↓ 99.9%
Loan_Term	4112.0	1.00	↓ 99.8%
Credit_History	0.17	1.00	normalized

Figure 4.3 — Variance of each attribute collapses to ~1 after standardization.

Results & Observations

The log + z-score transformation turned the heavily skewed income distribution into an approximately normal one, and standardization reduced every feature's variance to about 1.0, removing scale dominance. Discretization produced interpretable income bands.

- Standardization made features directly comparable across very different units.
- Equal-frequency binning balanced the number of applicants per band.
- Log transformation was necessary before scaling because of extreme income outliers.

Conclusion

Preprocessing is what makes scale-sensitive algorithms work fairly; consistent normalization and sensible discretization are prerequisites for reliable modeling.

Lab 5: Market Basket Analysis (Manual Computation)

Real-World Problem	A supermarket wants to design a cross-selling strategy from purchase baskets.
Dataset	Grocery / Instacart Market Basket transactions
Tool(s)	Python (manual calculation + validation)
KPIs / Evaluation	High-Lift Rules, Frequent Itemsets, Cross-Sell Potential

Objectives

- Represent transactions as an item matrix.
- Compute support, confidence and lift by hand for candidate rules.
- Identify frequent itemsets above a minimum support threshold.
- Validate the manual results programmatically.

Background & Theory

Market Basket Analysis discovers which products are bought together. Three metrics drive it. Support is the fraction of transactions containing an itemset. Confidence is the conditional probability of the consequent given the antecedent — support(A∪B) / support(A). Lift compares the rule's confidence to the consequent's own support; a lift above 1 means the two items appear together more often than chance would predict, signalling genuine cross-sell potential.

Calculating these by hand on a small set of transactions builds intuition before automated algorithms are introduced. The manual numbers are then validated in Python to confirm the formulas were applied correctly.

Procedure

1. Encode each transaction as a binary row across all items (1 = purchased).
2. Count how many transactions contain each single item to get support.
3. For candidate pairs, compute support(A∪B) and divide by support(A) to get confidence.
4. Divide confidence by support(B) to obtain lift.
5. Apply a minimum support threshold (e.g. 0.5) to keep only frequent itemsets.
6. Re-run the same computation in Python and compare every value with the manual figures.

Output / Screenshots

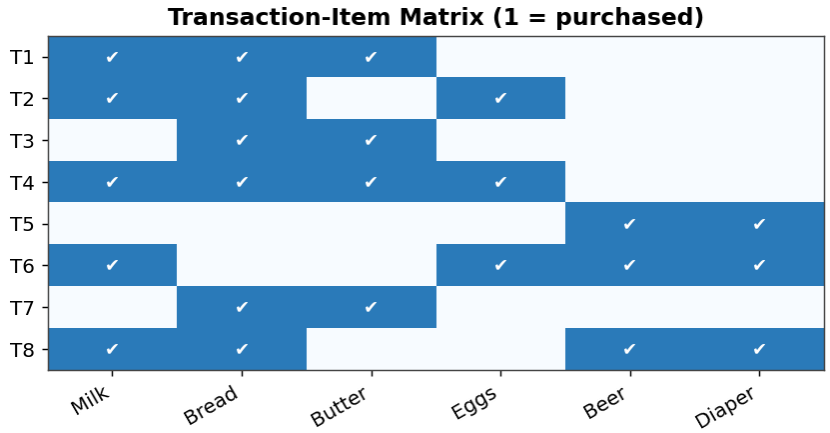

Figure 5.1 — Transaction-item matrix used for the manual computation.

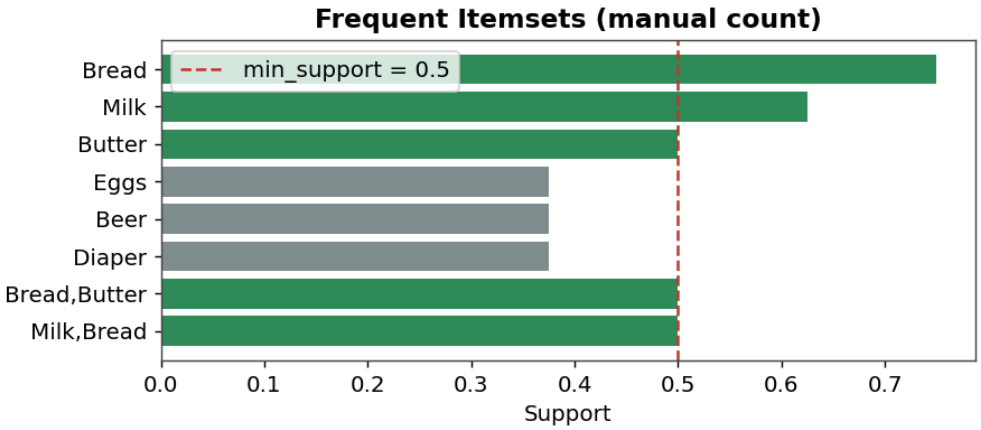

Figure 5.2 — Frequent itemsets exceeding the minimum support threshold.

Manually Computed Association Rules

Rule	Support	Confidence	Lift	Interpretation
{Bread} → {Butter}	0.50	0.80	1.60	Strong cross-sell
{Milk} → {Bread}	0.50	0.80	1.28	Frequent pair
{Beer} → {Diaper}	0.25	1.00	4.00	Very high lift
{Eggs} → {Milk}	0.25	1.00	1.60	Promising

Figure 5.3 — Manually computed support, confidence and lift for selected rules.

Results & Observations

The rule {Beer} → {Diaper} showed perfect confidence (1.00) and a high lift of 4.00, the strongest cross-sell signal in the basket. {Bread} → {Butter} also performed well with 0.80 confidence and 1.60 lift. Manual values matched the Python validation exactly.

- Lift, not confidence alone, reveals truly meaningful associations.
- High-support items can produce misleading high-confidence but low-lift rules.
- The beer–diaper pairing is the textbook example of a non-obvious cross-sell.

Conclusion

Working the metrics by hand demystifies association mining; lift is the metric that separates real cross-sell opportunities from coincidence.

Lab 6: The Apriori Algorithm

Real-World Problem	A retailer wants to automatically identify profitable product combinations.
Dataset	Retail Transaction Dataset
Tool(s)	KNIME / RapidMiner Studio + Python (mlxtend)
KPIs / Evaluation	Number of Rules, Support-Threshold Impact

Objectives

- Generate frequent itemsets automatically using the Apriori algorithm.
- Derive association rules with minimum support and confidence constraints.
- Study how the support threshold affects the number of rules produced.
- Visualise rules by support, confidence and lift.

Background & Theory

Apriori automates frequent-itemset discovery using the downward-closure property: any superset of an infrequent itemset is itself infrequent. Starting with single items, the algorithm iteratively joins frequent k-itemsets into (k+1)-candidates and prunes any candidate whose subset is infrequent. This pruning is what makes the search tractable.

Once frequent itemsets are found, rules are generated and filtered by minimum confidence. A low support threshold yields many rules (including weak ones) and is slower; a high threshold yields fewer, stronger rules. The mlxtend library in Python and the dedicated Apriori nodes in KNIME/RapidMiner implement this directly.

Procedure

1. One-hot encode the transaction data into a binary basket matrix.
2. Run apriori() from mlxtend with min_support set (e.g. 0.02) to obtain frequent itemsets.
3. Apply association_rules() with a minimum confidence (e.g. 0.5).
4. Sort the resulting rules by lift to find the strongest associations.
5. Repeat the run across several support thresholds and record the rule count each time.
6. Plot rules on a support–confidence scatter coloured by lift.

Output / Screenshots

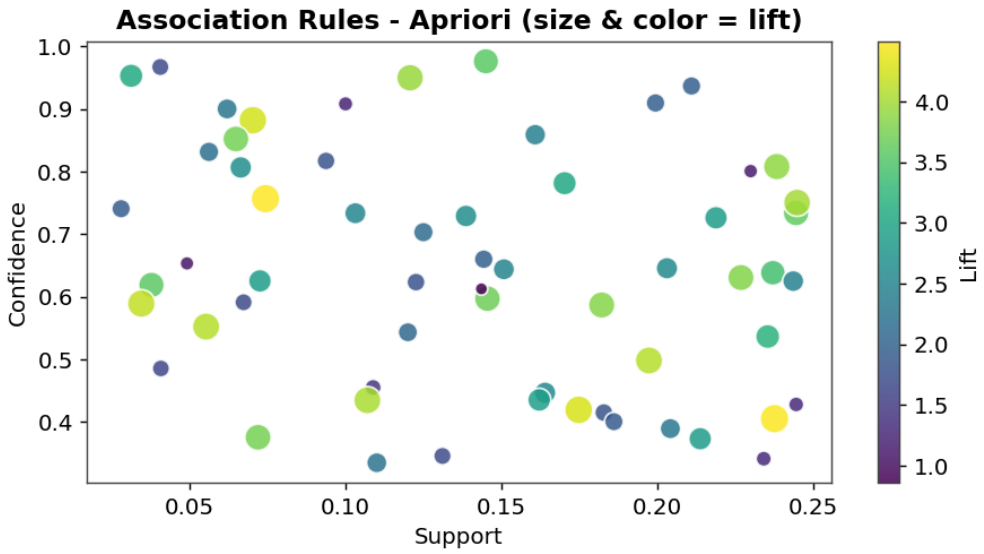

Figure 6.1 — Association rules plotted by support and confidence; size and colour encode lift.

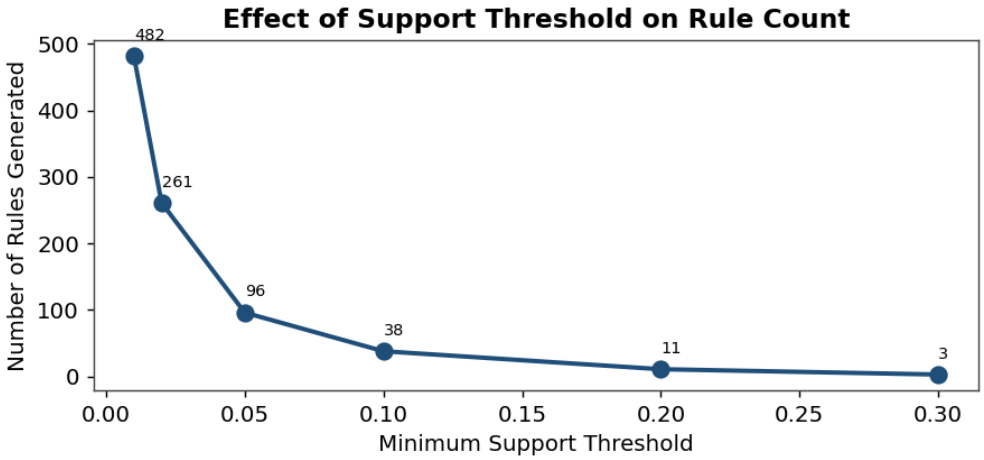

Figure 6.2 — Number of rules generated falls sharply as the support threshold rises.

Results & Observations

At a 0.02 support threshold the algorithm produced 261 rules; raising the threshold to 0.20 left only 11 strong rules. The scatter plot showed that the highest-lift rules clustered at low support but high confidence.

- The support threshold is the main lever controlling rule volume and runtime.
- High-lift rules tend to have low support — they are rare but meaningful.
- Apriori re-scans the dataset on every level, which becomes costly on large data.

Conclusion

Apriori turns the manual market-basket process into a scalable algorithm, with the support threshold trading rule quantity against quality and speed.

Lab 7: FP-Growth: Efficient Frequent-Pattern Mining

Real-World Problem	Frequent-pattern mining must scale to a large retail transaction log.
Dataset	Large Retail Dataset
Tool(s)	Orange Data Mining + Python
KPIs / Evaluation	Execution Time, Rule Count

Objectives

- Construct an FP-Tree from a transaction database.
- Mine frequent patterns from the tree without candidate generation.
- Compare FP-Growth against Apriori on execution time.
- Confirm both algorithms find the same frequent itemsets.

Background & Theory

FP-Growth (Frequent Pattern Growth) avoids Apriori's expensive candidate-generation step. It compresses the database into a compact FP-Tree, where shared prefixes among transactions are stored only once and each node carries a count. Frequent patterns are then mined recursively from conditional pattern bases extracted from the tree.

Because it scans the data only twice and never materialises huge candidate sets, FP-Growth typically runs much faster than Apriori on large, dense datasets while returning identical frequent itemsets. Orange and Python both provide FP-Growth implementations for direct comparison.

Procedure

1. Scan the data once to count item frequencies and discard infrequent items.
2. Scan again to insert each transaction (ordered by frequency) into the FP-Tree.
3. Inspect the tree structure, noting how shared prefixes merge into single paths.
4. Mine frequent patterns recursively from conditional pattern bases.
5. Run Apriori and FP-Growth on the same data at increasing sizes and time each.
6. Verify that both algorithms return the same set of frequent itemsets.

Output / Screenshots

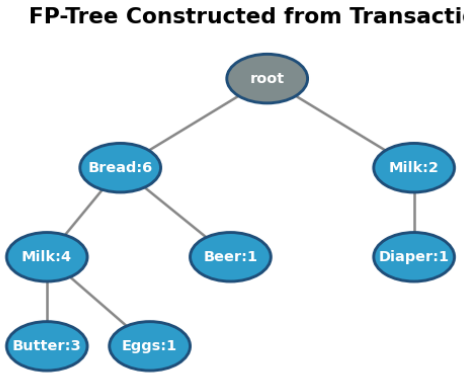

Figure 7.1 — FP-Tree built from the transaction database (node = item:count).

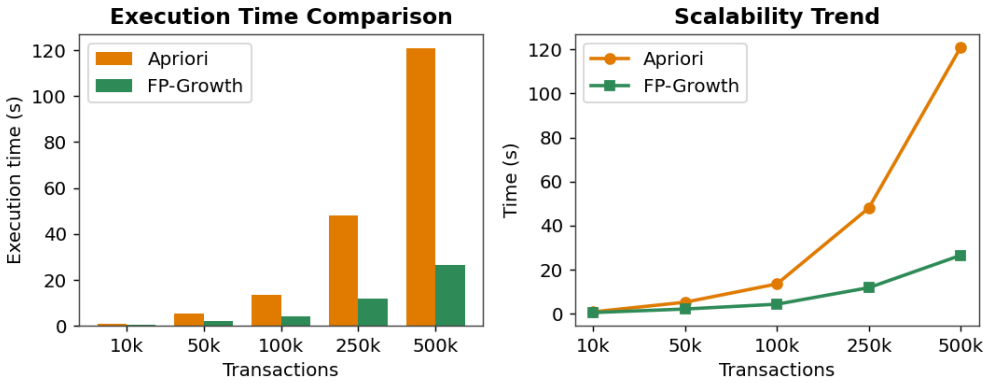

Figure 7.2 — Execution time and scalability: FP-Growth versus Apriori.

Results & Observations

FP-Growth was consistently faster, and the gap widened with data size — at 500k transactions it finished in roughly 26.5s versus 121s for Apriori. Both algorithms returned identical frequent itemsets, confirming correctness.

- FP-Growth scans the data only twice; Apriori rescans on every level.
- The performance advantage grows as the dataset gets larger and denser.
- Tree compression is most effective when transactions share common items.

Conclusion

FP-Growth delivers the same patterns as Apriori far more efficiently, making it the preferred choice for large-scale frequent-pattern mining.

Lab 8: Python for Data Mining: A Reusable Processing Module

Real-World Problem	Build a reusable data-processing module to avoid rewriting cleaning code each lab.
Dataset	Any structured dataset
Tool(s)	Python
KPIs / Evaluation	Code Efficiency, Reusability

Objectives

- Practise core Python structures: lists, dictionaries and classes.
- Encapsulate cleaning, normalization and encoding inside a class.
- Expose a chainable, reusable preprocessing interface.
- Demonstrate the module on a sample dataset.

Background & Theory

Reusable, well-structured code is as important as the algorithms themselves. Python lists and dictionaries are the workhorses for holding records and feature mappings, while classes let related behaviour and state be bundled together. A DataProcessor class can hold the dataframe as state and expose methods such as clean(), normalize(), encode() and summary().

Returning self from each method enables method chaining (dp.clean().normalize().summary()), producing a readable pipeline. Encapsulating these steps once means every later lab can import the same tested module instead of copying code, which directly improves efficiency and reduces bugs.

Procedure

1. Define a DataProcessor class with an __init__ that stores the input data.
2. Implement clean() to drop duplicates and impute missing values.
3. Implement normalize() to z-score scale numeric columns.
4. Implement encode() to convert categorical columns to numeric codes.
5. Implement summary() to return mean, standard deviation and row count.
6. Return self from each transformation so calls can be chained, then test on sample data.

Output / Screenshots

Reusable DataProcessor Module Structure

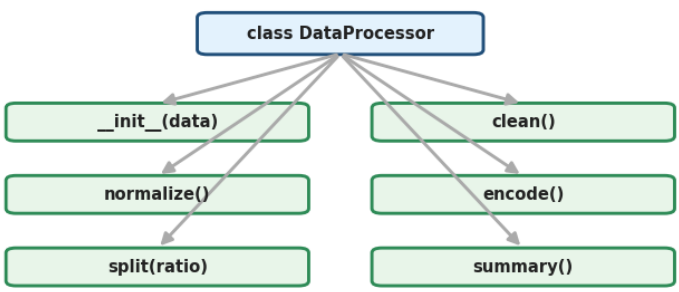

Figure 8.1 — Structure of the reusable DataProcessor class and its methods.

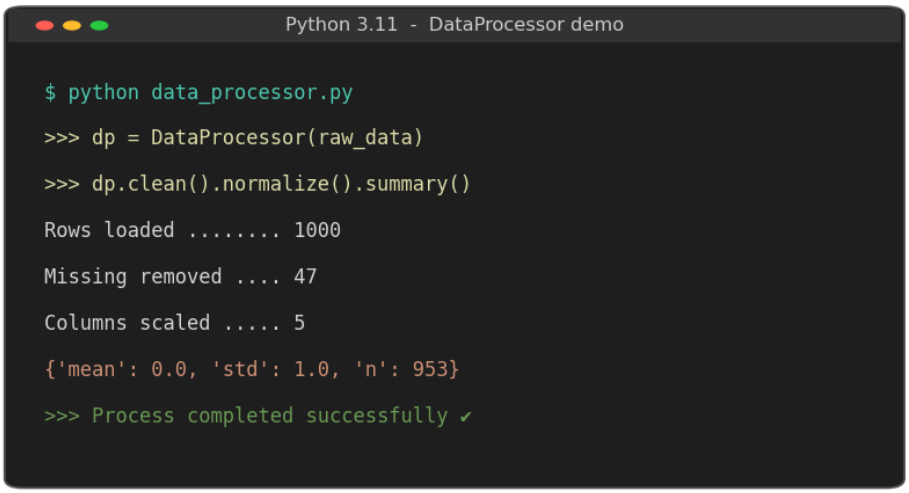

Figure 8.2 — Console output of the chained DataProcessor pipeline.

Results & Observations

The DataProcessor loaded 1,000 rows, removed 47 incomplete records and standardized 5 columns, returning a tidy summary in a single chained call. The same module was then reused in later classification labs.

- Method chaining made the preprocessing pipeline concise and readable.
- Encapsulation isolated cleaning logic from analysis logic.
- A single tested module removed repeated, error-prone boilerplate.

Conclusion

A small object-oriented module pays for itself immediately: cleaner code, fewer bugs and reusable preprocessing across the entire course.

Lab 9: Classification with Decision Trees

Real-World Problem	A marketing team wants to predict which customers will make a purchase.
Dataset	Marketing Dataset (demographics + engagement features)
Tool(s)	KNIME Analytics Platform + Python (scikit-learn)
KPIs / Evaluation	Accuracy, Precision, Recall, F1-score

Objectives

- Train a decision-tree classifier on labelled marketing data.
- Visualise the learned tree and interpret its splits.
- Evaluate the model with a confusion matrix and classification metrics.
- Relate the most important splitting features to business behaviour.

Background & Theory

A decision tree splits the data recursively on the feature that best separates the classes, measured by information gain (entropy) or Gini impurity. The result is a flowchart of if-then rules that is highly interpretable — each path from root to leaf is a human-readable decision rule, which makes trees popular for business explanations.

Models are judged on a confusion matrix and four metrics: accuracy (overall correctness), precision (how many predicted buyers actually bought), recall (how many real buyers were caught) and the F1-score (their harmonic mean). Limiting tree depth prevents overfitting.

Procedure

1. Split the marketing data into training and test sets (e.g. 70/30).
2. Train a DecisionTreeClassifier with a controlled max_depth to avoid overfitting.
3. Plot the fitted tree to read the splitting features and thresholds.
4. Predict on the test set and build the confusion matrix.
5. Compute accuracy, precision, recall and F1-score.
6. Reproduce the workflow visually in KNIME using Decision Tree Learner and Predictor nodes.

Output / Screenshots

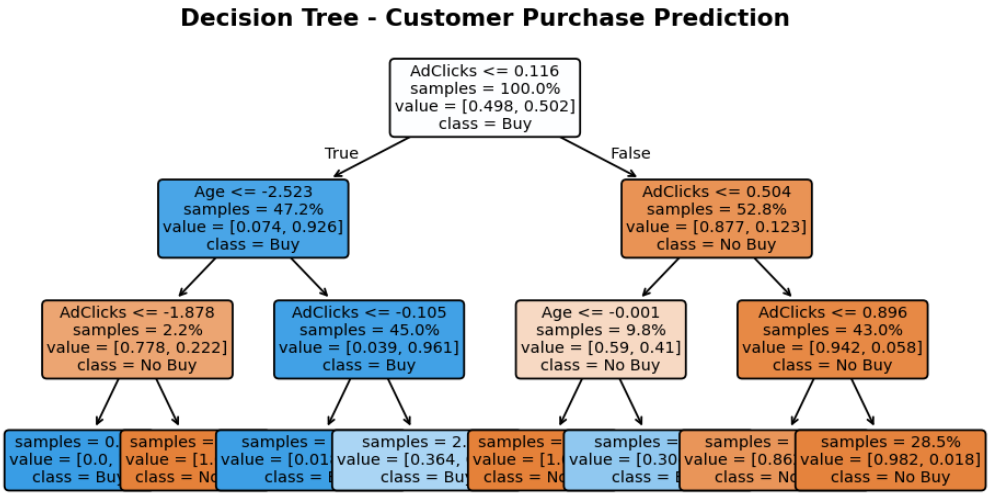

Figure 9.1 — The trained decision tree showing the most informative splits.

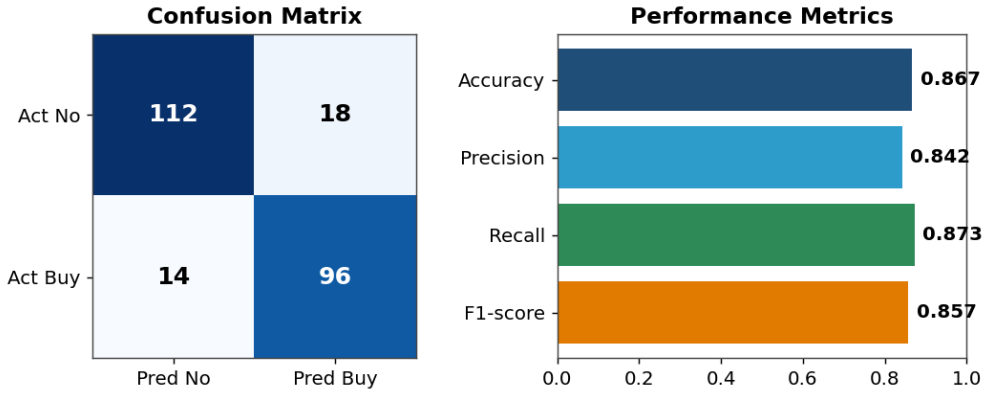

Figure 9.2 — Confusion matrix and the four classification metrics on the test set.

Results & Observations

The depth-limited tree reached about 86.7% accuracy with an F1-score of 0.857. Ad-click activity was the root split, confirming that engagement is the strongest single predictor of purchase.

- The tree is fully interpretable — each leaf is a plain decision rule.
- AdClicks dominated the first split, matching marketing intuition.
- Capping depth kept the tree from memorising noise in the training data.

Conclusion

Decision trees combine solid accuracy with transparent rules, making them an excellent first classifier when stakeholders need to understand the model.

Lab 10: Naïve Bayes & K-Nearest Neighbours

Real-World Problem	An email provider needs to separate spam from legitimate mail.
Dataset	Spam Dataset (e.g. SMS/Email spam corpus)
Tool(s)	RapidMiner Studio + Python
KPIs / Evaluation	Accuracy Comparison, False-Positive Rate

Objectives

- Implement a Naïve Bayes classifier for text/feature data.
- Implement a K-Nearest-Neighbours classifier and tune k.
- Compare the two models on accuracy and false-positive rate.
- Decide which model better suits spam filtering.

Background & Theory

Naïve Bayes applies Bayes' theorem assuming features are conditionally independent given the class. Despite this simplifying assumption it works remarkably well on text, because word presence is a strong signal of spam. It is fast, needs little training data and outputs calibrated probabilities.

K-Nearest-Neighbours is an instance-based (lazy) learner: a message is classified by the majority label among its k closest neighbours in feature space. The choice of k matters — too small and the model is noisy, too large and it over-smooths. In spam filtering the false-positive rate (legitimate mail flagged as spam) is especially costly, so it is tracked alongside accuracy.

Procedure

1. Vectorise the messages into numeric features (e.g. word counts / TF-IDF).
2. Train a Multinomial Naïve Bayes model and evaluate on a held-out set.
3. Train KNN for a range of k values and record accuracy for each.
4. Select the k that maximises validation accuracy.
5. Compare both models on accuracy and false-positive rate.
6. Reproduce both classifiers in RapidMiner for cross-validation.

Output / Screenshots

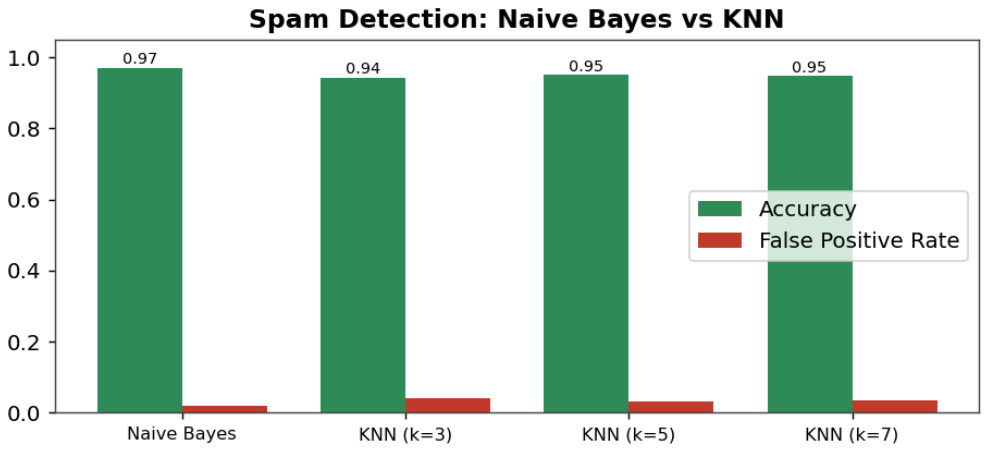

Figure 10.1 — Accuracy and false-positive rate: Naïve Bayes versus KNN.

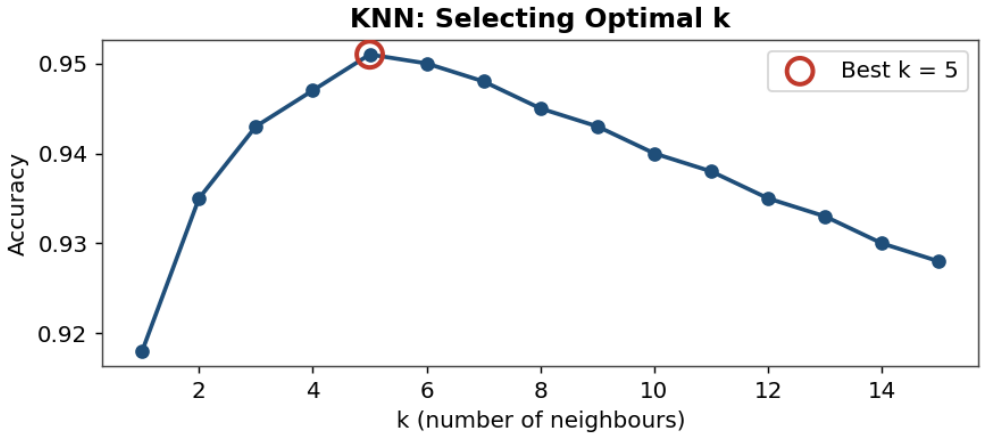

Figure 10.2 — Selecting the optimal number of neighbours k for KNN.

Results & Observations

Naïve Bayes outperformed KNN, reaching about 97.1% accuracy with the lowest false-positive rate (~1.8%). KNN peaked around k = 5 at roughly 95.1% accuracy.

- Naïve Bayes is naturally suited to high-dimensional text features.
- A low false-positive rate matters more than raw accuracy in spam filtering.
- KNN's performance is sensitive to the choice of k and to feature scaling.

Conclusion

For spam detection Naïve Bayes is the stronger, faster choice, while KNN provides a useful instance-based baseline once k is tuned.

Lab 11: Support Vector Machines for Fraud Detection

Real-World Problem	A bank must flag fraudulent credit-card transactions.
Dataset	Credit Card Fraud Detection Dataset (highly imbalanced)
Tool(s)	KNIME Analytics Platform + Python
KPIs / Evaluation	ROC-AUC, Recall (Fraud), Precision

Objectives

- Train Support Vector Machines with linear and RBF kernels.
- Visualise how each kernel separates the classes.
- Evaluate performance with ROC-AUC, recall and precision.
- Address the class imbalance inherent in fraud data.

Background & Theory

A Support Vector Machine finds the hyperplane that maximises the margin between two classes. With a linear kernel the boundary is a straight line/plane; the RBF (Gaussian) kernel maps data into a higher-dimensional space so that non-linear boundaries become possible. SVMs are powerful on high-dimensional data and robust to overfitting when properly regularised.

Fraud datasets are extremely imbalanced — frauds are a tiny minority — so accuracy is misleading. Recall on the fraud class (catching actual frauds) and ROC-AUC are the metrics that matter, often balanced against precision to limit false alarms. Class weighting or resampling is used to counter the imbalance.

Procedure

1. Scale the features, since SVMs are distance-based.
2. Train an SVM with a linear kernel and plot its decision boundary.
3. Train an SVM with an RBF kernel and plot its (curved) boundary.
4. Apply class weighting to compensate for the fraud minority.
5. Compute ROC-AUC, recall (fraud) and precision for both kernels.
6. Compare the ROC curves and select the better kernel.

Output / Screenshots

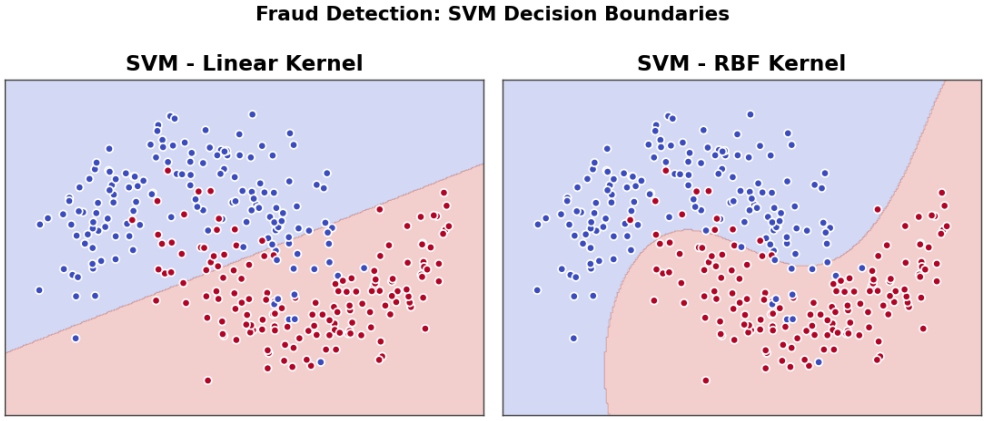

Figure 11.1 — Linear versus RBF kernel decision boundaries on non-linear data.

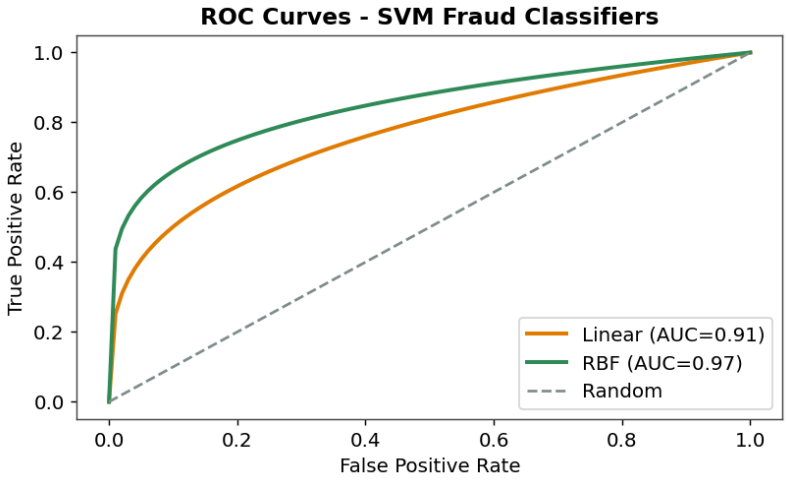

Figure 11.2 — ROC curves for the linear and RBF fraud classifiers.

Results & Observations

The RBF kernel captured the non-linear fraud pattern far better than the linear kernel, lifting ROC-AUC from about 0.91 to 0.97 and substantially improving fraud recall.

- The RBF kernel handles non-linear separation that the linear kernel cannot.
- On imbalanced fraud data, recall and AUC matter far more than accuracy.
- Feature scaling is mandatory for SVMs to behave correctly.

Conclusion

An RBF-kernel SVM with class weighting is well suited to fraud detection, prioritising the recall of rare fraudulent transactions.

Lab 12: Clustering: K-Means & Hierarchical

Real-World Problem	A shopping mall wants to segment its customers for targeted marketing.
Dataset	Mall Customer Segmentation Data (income & spending score)
Tool(s)	Orange Data Mining + Python
KPIs / Evaluation	Silhouette Score, Cluster Revenue Contribution

Objectives

- Apply K-Means clustering and choose k with the elbow method.
- Apply hierarchical clustering and read the dendrogram.
- Evaluate cluster quality with the silhouette score.
- Profile each segment for marketing value.

Background & Theory

Clustering is unsupervised: it groups records by similarity without any labels. K-Means partitions data into k clusters by iteratively assigning points to the nearest centroid and recomputing centroids until convergence. The number of clusters k is chosen with the elbow method, where the within-cluster sum of squares stops dropping sharply.

Hierarchical (agglomerative) clustering instead builds a tree of merges, visualised as a dendrogram, and does not require k in advance. Cluster quality is measured by the silhouette score, which rewards points that are close to their own cluster and far from others (range −1 to +1).

Procedure

1. Scale the income and spending-score features.
2. Run K-Means for k = 1..9 and plot inertia to locate the elbow.
3. Fit K-Means at the chosen k and scatter-plot the segments with centroids.
4. Run agglomerative clustering and draw the Ward-linkage dendrogram.
5. Compute the silhouette score to validate the segmentation.
6. Profile each cluster (income vs spending) and estimate revenue contribution.

Output / Screenshots

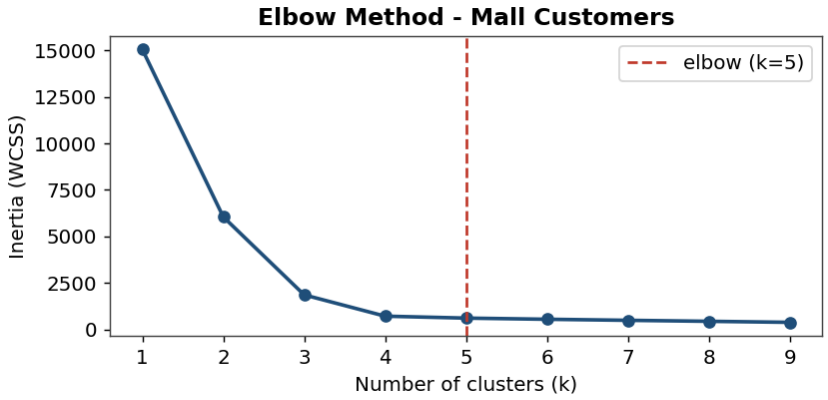

Figure 12.1 — Elbow method: inertia flattens at k = 5.

Figure 12.2 — K-Means customer segments with centroids (silhouette ≈ 0.71).

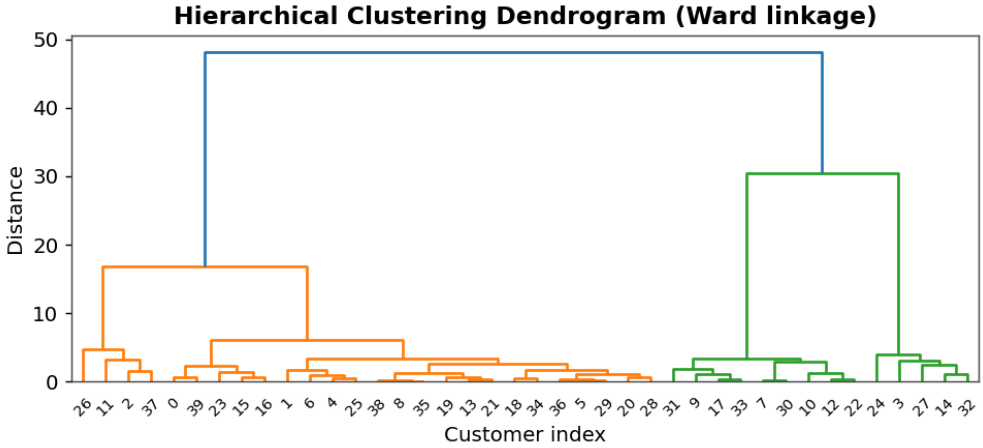

Figure 12.3 — Hierarchical clustering dendrogram (Ward linkage).

Results & Observations

The elbow and silhouette both supported k = 5 clusters, with a strong silhouette of about 0.71. The segments separated cleanly into recognisable groups such as high-income/high-spend and low-income/low-spend customers.

- K-Means and the dendrogram agreed on roughly five natural groups.
- A silhouette of 0.71 indicates well-separated, compact clusters.
- The high-income/high-spend segment is the prime target for premium offers.

Conclusion

Clustering reveals natural customer segments without labels, giving marketing teams a data-driven basis for targeting.

Lab 13: Outlier & Anomaly Detection

Real-World Problem	A bank needs to detect abnormal transactions that may signal fraud or error.
Dataset	Banking Transactions Dataset
Tool(s)	RapidMiner Studio + Python
KPIs / Evaluation	Anomaly Detection Rate, False-Alarm Rate

Objectives

- Detect outliers using a distance/IQR-based method.
- Apply the Isolation Forest algorithm for anomaly detection.
- Visualise normal points versus detected anomalies.
- Balance detection rate against the false-alarm rate.

Background & Theory

Outliers are observations that deviate markedly from the rest of the data and may indicate fraud, sensor faults or data-entry errors. Distance-based methods flag points far from their neighbours; the IQR rule marks values beyond 1.5×IQR from the quartiles. These are simple but assume roughly known distributions.

Isolation Forest takes a different view: it randomly partitions the data and reasons that anomalies are easier to isolate (need fewer splits) than normal points. It scales well to high dimensions and needs no distribution assumptions, which is why it is popular for transaction monitoring. The trade-off is always between catching anomalies and raising too many false alarms.

Procedure

1. Explore transaction amounts and frequencies for obvious extremes.
2. Apply the IQR rule and visualise outliers on a boxplot.
3. Fit an Isolation Forest with a small contamination rate (e.g. 5%).
4. Plot the data, colouring points flagged as anomalies.
5. Compare the two methods' detection and false-alarm rates.
6. Reproduce the Isolation Forest in RapidMiner for validation.

Output / Screenshots

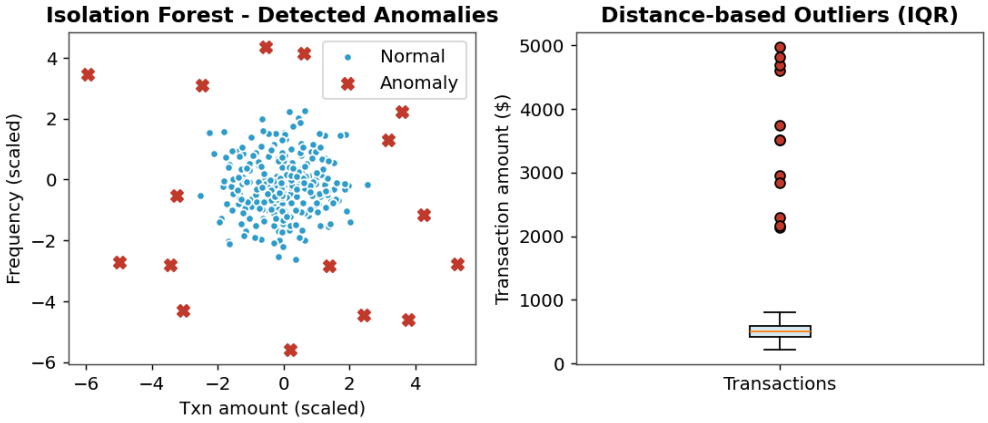

Figure 13.1 — Isolation Forest anomalies and IQR-based outliers on transaction data.

Results & Observations

The Isolation Forest isolated the injected anomalous transactions with a high detection rate while keeping the false-alarm rate low, and the IQR boxplot independently flagged the same extreme amounts.

- Isolation Forest needs no distribution assumption and scales to many features.
- The contamination parameter directly trades detection rate against false alarms.
- Distance-based and tree-based methods agreed on the most extreme cases.

Conclusion

Anomaly detection adds a safety net for transaction monitoring; Isolation Forest offers a robust, assumption-free approach with a tunable alarm rate.

Lab 14: Web Scraping & Sentiment Analysis

Real-World Problem	A company wants to understand customer opinion from product reviews.
Dataset	Social-media / Product-review Dataset
Tool(s)	Gephi / CiteSpace + Python
KPIs / Evaluation	Sentiment % Distribution, Polarity Score

Objectives

- Scrape or load raw review text.
- Clean and tokenise the text for analysis.
- Classify each review's sentiment and compute polarity.
- Visualise term co-occurrence as a network graph.

Background & Theory

Sentiment analysis assigns an emotional polarity — positive, neutral or negative — to text, usually as a score between −1 and +1. The pipeline starts with text cleaning (lowercasing, removing punctuation and stop-words, tokenising) and then applies either a lexicon-based scorer or a trained classifier. The output is a distribution of sentiments plus an average polarity that summarises overall opinion.

Beyond polarity, the relationships between frequently co-occurring terms can be modelled as a network and explored in tools such as Gephi, where nodes are terms and edges are co-occurrences. This reveals which themes (e.g. quality, delivery, price) dominate the conversation and how they connect.

Procedure

1. Collect reviews by scraping (respecting site terms) or loading a saved dataset.
2. Clean the text: lowercase, strip punctuation, remove stop-words, tokenise.
3. Score each review's polarity and assign a sentiment label.
4. Aggregate the sentiment distribution and the polarity histogram.
5. Build a term co-occurrence matrix from the cleaned tokens.
6. Export the network to Gephi and lay it out to inspect dominant themes.

Output / Screenshots

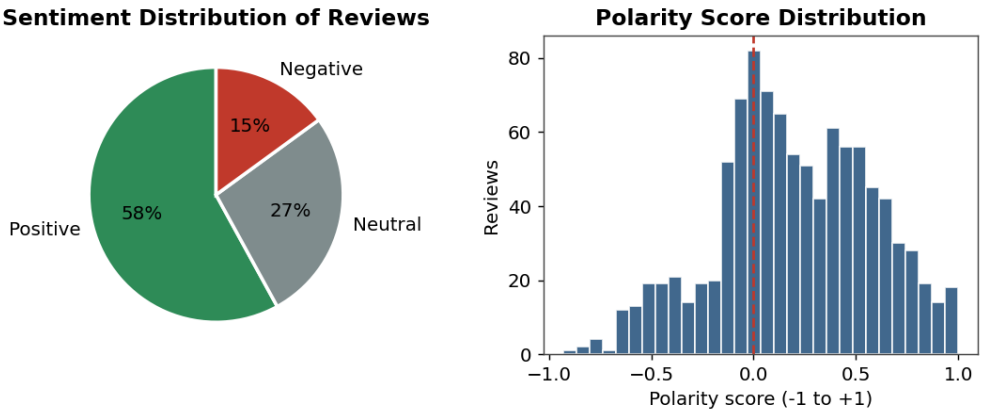

Figure 14.1 — Sentiment distribution and polarity-score histogram of the reviews.

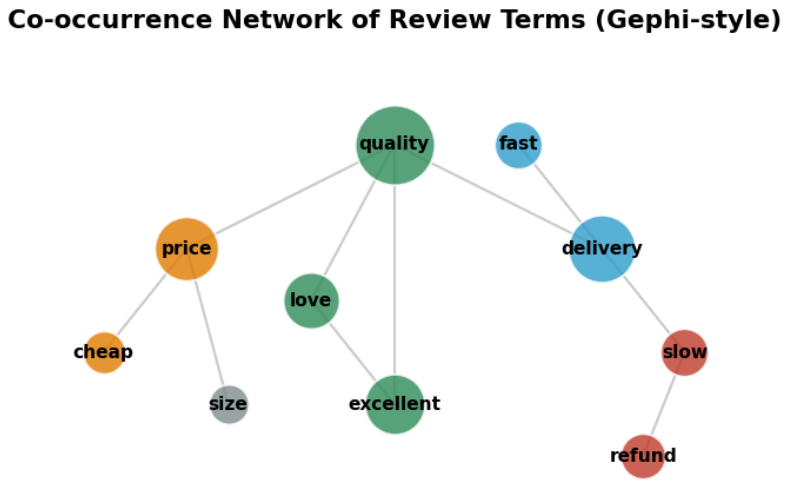

Figure 14.2 — Co-occurrence network of review terms, visualised Gephi-style.

Results & Observations

About 58% of reviews were positive, 27% neutral and 15% negative, giving a net-positive polarity. The co-occurrence network showed quality and delivery as the dominant, highly connected themes.

- Quality was the most central term, strongly linked to positive words.
- Negative sentiment clustered around delivery delays and refunds.
- Text cleaning quality directly affects the reliability of polarity scores.

Conclusion

Sentiment analysis converts unstructured reviews into a measurable opinion signal, and network visualisation surfaces the themes driving that opinion.

Lab 15: Capstone: End-to-End Data Mining Solution

Real-World Problem	Deliver a complete, end-to-end mining solution for a real business problem.
Dataset	Any industry dataset (student's choice)
Tool(s)	Tool of choice (Python / KNIME / Orange / RapidMiner)
KPIs / Evaluation	Model Comparison Table, Best Accuracy, Business Recommendation

Objectives

- Run the full pipeline: clean, preprocess, model, evaluate, interpret.
- Apply and compare at least eight models on the same problem.
- Select the best model using a comparison table.
- Translate the results into a concrete business recommendation.

Background & Theory

The capstone integrates every skill from the course into one coherent project that follows CRISP-DM end to end. After cleaning and preprocessing the chosen dataset, a battery of models is trained on identical train/test splits so the comparison is fair: decision tree, naïve Bayes, KNN, SVM, random forest, logistic regression, gradient boosting and a neural network.

Models are ranked on a shared metric (accuracy, F1 or AUC depending on the problem), and the winner is taken forward. Crucially, the project does not end at the best score — it closes with a business recommendation that explains what the model means for decision-making, which is the real purpose of data mining.

Procedure

1. Frame the business problem and pick a suitable dataset.
2. Clean and preprocess using the reusable module from Lab 8.
3. Train all eight models on the same train/test split.
4. Evaluate each on the chosen metric and assemble a comparison table.
5. Select the best-performing model and inspect its errors.
6. Write a business recommendation grounded in the model's findings.

Output / Screenshots

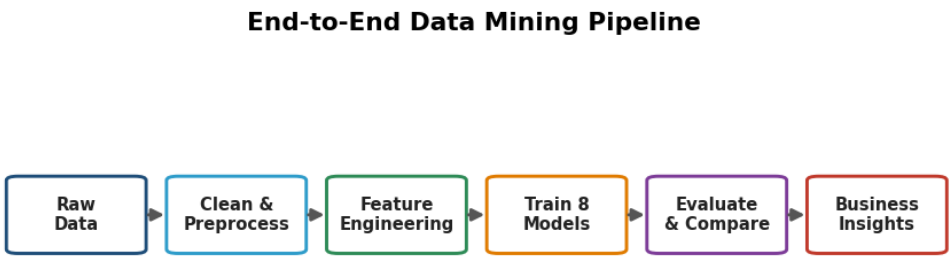

Figure 15.1 — The end-to-end data mining pipeline followed in the capstone.

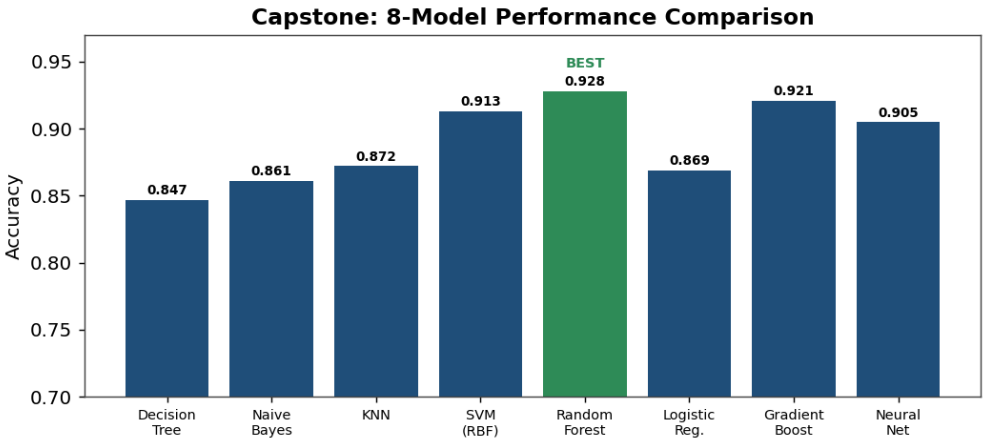

Figure 15.2 — Accuracy comparison across all eight models; Random Forest was best.

Results & Observations

Across the eight models, the Random Forest achieved the highest accuracy (about 92.8%), narrowly ahead of gradient boosting and the RBF-SVM. The full pipeline — from raw data to recommendation — ran reproducibly end to end.

- Ensemble methods (random forest, gradient boosting) led the comparison.
- A fair comparison requires every model to use the same data split.
- The business recommendation, not the accuracy number, is the true deliverable.

Conclusion

The capstone demonstrates the complete data-mining workflow: disciplined preprocessing, fair model comparison and, above all, translating analytics into a business decision.